

Seguridad: Cifrado de Contraseñas con SHA2 en MySQL

Contexto

Luego de haber trabajado con:

- Creación de usuarios y roles (CREATE USER, GRANT),
- Control de accesos (GRANT, REVOKE),
- Auditorías con triggers o logs,

Ahora pasamos a un aspecto esencial: **la protección de los datos sensibles**, como contraseñas. En este caso, lo hacemos con **cifrado unidireccional (hashing)**.

Simulación: Registro y Login usando SHA2

1. Crear la tabla de usuarios

```
CREATE TABLE Usuarios (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(100),  
  correo VARCHAR(100),  
  password VARCHAR(256)  
);
```

2. Insertar un usuario con contraseña cifrada

Usamos la función SHA2('texto', 256) para generar un hash de 256 bits del texto plano.

```
INSERT INTO Usuarios(nombre, correo, password)  
VALUES ('Ana', 'ana@mail.com', SHA2('secreta123', 256));
```

Esto es solo una **simulación didáctica**. En entornos reales se usa una función como bcrypt, argon2, o un sistema externo (como autenticación desde backend).

3. Simulación de login (validación)

Comprobamos si el correo y la contraseña ingresada (cifrada en el momento) coinciden con un registro:

```
SELECT * FROM Usuarios  
WHERE correo = 'ana@mail.com'  
AND password = SHA2('secreta123', 256);
```

- Cómo almacenar contraseñas cifradas (no en texto plano).

- Cómo validar al usuario sin desencriptar la contraseña (ya que los hashes no se desencriptan).
- Cómo integrar funciones hash nativas de MySQL (SHA2, MD5, etc.) en consultas SQL.

4. Opcional: Validar que el hash sea correcto

-- Ver cómo se ve el hash

```
SELECT SHA2('secreta123', 256) AS hash;
```

-- Ver contenido completo

```
SELECT * FROM Usuarios;
```

Buenas prácticas

1. No se debe almacenar contraseñas sin cifrado.
2. No se deben mostrar los hashes en interfaces o formularios.
3. En producción, se usan funciones con **salt** y algoritmos más seguros (como bcrypt, no disponible en MySQL puro).

Ejercicio de Simulación de Cifrado de Contraseñas con SHA2 en MySQL

-- **Paso 1:** Eliminamos la tabla si ya existía (para pruebas)

```
DROP TABLE IF EXISTS Usuarios;
```

-- **Paso 2:** Creamos la tabla de usuarios con campo para contraseña cifrada

```
CREATE TABLE Usuarios (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(100),  
  correo VARCHAR(100),  
  password CHAR(64) -- almacenará el hash SHA-256, que siempre devuelve  
  64 caracteres hexadecimales, usando CHAR(64) es más preciso.  
);
```

-- **Paso 3:** Insertamos un usuario con la contraseña cifrada usando SHA2

-- **IMPORTANTE:** SHA2('texto', 256) genera un hash de 64 caracteres

```
INSERT INTO Usuarios(nombre, correo, password)  
VALUES ('Ana', 'ana@mail.com', SHA2('secreta123', 256));
```

-- **Paso 4:** Visualizamos la tabla para comprobar el hash generado

```
SELECT * FROM Usuarios;
```

-- **Paso 5:** Simulamos un intento de login (correo + contraseña cifrada)

-- Si el correo y la contraseña cifrada coinciden, devuelve el usuario

```
SELECT * FROM Usuarios  
WHERE correo = 'ana@mail.com'  
AND password = SHA2('secreta123', 256);
```

-- Este ejemplo es solo didáctico. En producción se usa bcrypt, Argon2, etc.

-- **Paso 6:** Simulamos un intento de login con contraseña incorrecta

-- No debería devolver ningún resultado

```
SELECT * FROM Usuarios  
WHERE correo = 'ana@mail.com'  
AND password = SHA2('claveIncorrecta', 256);  
-- Al comparar hashes, si la contraseña ingresada es incorrecta, MySQL no  
devuelve ningún registro, ya que el hash generado no coincide con el  
almacenado.
```

Comentarios adicionales

- **Qué es SHA2:** función hash segura de la familia SHA (Secure Hash Algorithm). SHA2(..., 256) es SHA-256.
- **Por qué es unidireccional:** No se puede revertir el hash para obtener la contraseña original.
- **Limitaciones:** SHA2 no es suficiente por sí solo en producción (se requieren salts, algoritmos más robustos como bcrypt).
- **Simulación de login:** Este ejemplo permite mostrar cómo se comparan las credenciales cifradas sin descifrarlas.

Ejercicios con cifrado de usuarios

EJERCICIO 1 – Oficina Pública de Reclamos: Registro de Usuarios con Cifrado SHA2

Enunciado:

En una oficina pública de atención al ciudadano se implementa un sistema de registro de cuentas para que los ciudadanos puedan hacer reclamos. Las contraseñas se almacenan cifradas con SHA2.

Script paso a paso:

```
-- Paso 1: Crear base de datos  
DROP DATABASE IF EXISTS OficinaReclamos;  
CREATE DATABASE OficinaReclamos;  
USE OficinaReclamos;
```

```
-- Paso 2: Crear tabla de usuarios  
CREATE TABLE Ciudadanos (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(100),  
    dni VARCHAR(20) UNIQUE,  
    correo VARCHAR(100) UNIQUE,  
    password CHAR(64) -- Hash SHA-256  
);
```

```
-- Paso 3: Insertar un ciudadano con contraseña cifrada  
INSERT INTO Ciudadanos(nombre, dni, correo, password)  
VALUES ('Lucía Pérez', '12345678', 'lucia@mail.com', SHA2('lucia123', 256));
```

-- **Paso 4:** Verificar datos ingresados

```
SELECT * FROM Ciudadanos;
```

-- **Paso 5:** Simulación de login exitoso

```
SELECT * FROM Ciudadanos
```

```
WHERE correo = 'lucia@mail.com'
```

```
AND password = SHA2('lucia123', 256);
```

-- **Paso 6:** Simulación de login fallido

```
SELECT * FROM Ciudadanos
```

```
WHERE correo = 'lucia@mail.com'
```

```
AND password = SHA2('claveIncorrecta', 256);
```

Observaciones:

- 1) Se usa CHAR(64) para almacenar el hash.
- 2) El hash se genera al insertar la contraseña.
- 3) Login exitoso = combinación exacta de correo + hash.
- 4) Login fallido = contraseña incorrecta → no devuelve nada.

EJERCICIO 2 – Empresa de Producción: Cuentas de Empleados con Cifrado SHA2

Enunciado:

Una empresa de manufactura necesita registrar el acceso de su personal técnico y de ingeniería. Cada usuario tendrá su contraseña cifrada con SHA2. Se simulan distintos roles, aunque no se aplican restricciones todavía.

Objetivo del sistema

Administrar accesos y funciones para empleados según su **rol** dentro de la empresa: operarios, ingenieros, técnicos en seguridad y empaquetadores.

Roles definidos

- 1) Operario
- 2) Ingeniero de proceso
- 3) Técnico en Seguridad
- 4) Empaquetador

Restricciones

Rol	Permisos necesarios
Operario	Ver tareas asignadas
Ingeniero de Proceso	Ver y modificar procesos
Técnico en Seguridad	Ver alertas y registrar incidentes
Empaquetador	Ver lista de productos a embalar

Estructura de tablas

- 1) **Roles:** contiene los distintos roles.

- 2) **Permisos:** lista de acciones disponibles en el sistema.
- 3) **RolPermiso:** tabla intermedia que relaciona qué rol tiene qué permiso (relación muchos a muchos).

Código SQL paso a paso:

-- Paso 1: Crear base de datos

```
DROP DATABASE IF EXISTS ProduccionIndustrial;
```

```
CREATE DATABASE ProduccionIndustrial;
```

```
USE ProduccionIndustrial;
```

-- Paso 2: Crear tabla de roles

```
CREATE TABLE Roles (  
    id_rol INT AUTO_INCREMENT PRIMARY KEY,  
    nombre_rol VARCHAR(50)  
);
```

-- Paso 3 Insertar los roles

```
INSERT INTO Roles(nombre_rol) VALUES  
( 'Operario'),  
( 'Ingeniero de Proceso'),  
( 'Técnico en Seguridad'),  
( 'Empaquetador');
```

-- Paso 4: Crear tabla de empleados

```
CREATE TABLE Empleados (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(100),  
    correo VARCHAR(100) UNIQUE,  
    password CHAR(64),  
    id_rol INT,  
    FOREIGN KEY (id_rol) REFERENCES Roles(id_rol)  
);
```

-- Paso 5: Insertar varios empleados con contraseña cifrada y rol

```
INSERT INTO Empleados(nombre, correo, password, id_rol) VALUES  
( 'Juan Rodríguez', 'juan@fabrica.com', SHA2('juan123', 256), 1),  
( 'María Gómez', 'maria@fabrica.com', SHA2('maria456', 256), 2),  
( 'Carlos Díaz', 'carlos@fabrica.com', SHA2('seguridad789', 256), 3),  
( 'Laura Fernández', 'laura@fabrica.com', SHA2('laura321', 256), 4);
```

-- Paso 6: Crear tabla de permisos

```
CREATE TABLE Permisos (  
    id_permiso INT AUTO_INCREMENT PRIMARY KEY,  
    descripcion VARCHAR(100)  
);
```

-- Paso 7 Insertar permisos

```
INSERT INTO Permisos(descripcion) VALUES  
( 'Ver tareas asignadas'),  
( 'Modificar procesos'),  
( 'Ver alertas'),  
( 'Registrar incidentes');
```

('Ver productos a embalar');
Visualizar empleados
SELECT * FROM Empleados;

-- Paso 8: Crear tabla intermedia RolPermiso

```
CREATE TABLE RolPermiso (  
    id_rol INT,  
    id_permiso INT,  
    FOREIGN KEY (id_rol) REFERENCES Roles(id_rol),  
    FOREIGN KEY (id_permiso) REFERENCES Permisos(id_permiso)  
);
```

-- Paso 9: Asignar permisos a roles

-- Operario

```
INSERT INTO RolPermiso(id_rol, id_permiso) VALUES (1, 1);
```

-- Ingeniero de Proceso

```
INSERT INTO RolPermiso(id_rol, id_permiso) VALUES (2, 1), (2, 2);
```

-- Técnico en Seguridad

```
INSERT INTO RolPermiso(id_rol, id_permiso) VALUES (3, 3), (3, 4);
```

-- Empaquetador

```
INSERT INTO RolPermiso(id_rol, id_permiso) VALUES (4, 5);
```

-- Paso 10: Ejemplo concreto de uso

-- Consultar qué permisos tiene un usuario (por ejemplo María)

```
SELECT e.nombre AS empleado, r.nombre_rol, p.descripcion AS permiso  
FROM Empleados e  
JOIN Roles r ON e.id_rol = r.id_rol  
JOIN RolPermiso rp ON r.id_rol = rp.id_rol  
JOIN Permisos p ON rp.id_permiso = p.id_permiso  
WHERE e.correo = 'maria@fabrica.com';
```

Resultado esperado

Empleado	Nombre_rol	Permiso
María Gómez	Ingeniero de Proceso	Ver tareas asignadas
María Gomez	Ingeniero de Proceso	Modificar procesos

-- Paso 11: Simulación de login correcto

```
SELECT e.id, e.nombre, r.nombre_rol  
FROM Empleados e  
JOIN Roles r ON e.id_rol = r.id_rol  
WHERE e.correo = 'maria@fabrica.com'  
AND e.password = SHA2('maria456', 256);
```

-- Esto devuelve a María y su rol si la contraseña es correcta

-- Paso 12: Simulación de login fallido

```
SELECT e.id, e.nombre, r.nombre_rol  
FROM Empleados e  
JOIN Roles r ON e.id_rol = r.id_rol
```

```
WHERE e.correo = 'carlos@fabrica.com'  
AND e.password = SHA2('claveMala', 256);
```

Esto no devuelve nada, simulando un intento fallido.

Extensión

1. Control de acceso por rol
Ya esta armado con:
Roles
Permisos
RolPermiso
Y una consulta que muestra los permisos de cada usuarios según su rol.
2. Auditoría de Accesos

crear una tabla LogAccesos y un TRIGGER para registrar automáticamente:

- 1) Intentos exitosos o fallidos de login
- 2) Fecha y hora
- 3) Usuario (si coincide el correo)

3. Estado activo/inactivo en empleados
-- Agregamos una columna en Empleados→
ALTER TABLE Empleados ADD estado ENUM('activo', 'inactivo')
DEFAULT 'activo';

4. Modificamos la consulta de login
-- Solo permite login si el estado es 'activo'
SELECT e.id, e.nombre, r.nombre_rol
FROM Empleados e
JOIN Roles r ON e.id_rol = r.id_rol
WHERE e.correo = 'maria@fabrica.com'
AND e.password = SHA2('maria456', 256)
AND e.estado = 'activo';

```
-- Paso 13: Crear tabla de log de accesos  
CREATE TABLE LogAccesos (  
    id_log INT AUTO_INCREMENT PRIMARY KEY,  
    correo VARCHAR(100),  
    resultado VARCHAR(10), -- 'exitoso' o 'fallido'  
    fecha_hora DATETIME DEFAULT NOW()  
);
```

```
-- Paso 14: Crear procedimiento almacenado para login  
DELIMITER $$
```

```
CREATE PROCEDURE LoginUsuario(  
    IN p_correo VARCHAR(100),  
    IN p_password VARCHAR(100)  
)  
BEGIN
```

```
DECLARE v_contador INT;
```

```
-- Contamos si existe un usuario con ese correo, contraseña y estado activo
```

```
SELECT COUNT(*) INTO v_contador  
FROM Empleados  
WHERE correo = p_correo  
AND password = SHA2(p_password, 256)  
AND estado = 'activo';
```

```
-- Registramos el intento en el log  
INSERT INTO LogAccesos(correo, resultado)  
VALUES (p_correo, IF(v_contador > 0, 'exitoso', 'fallido'));
```

```
-- Mostramos el resultado al usuario  
IF v_contador > 0 THEN  
    SELECT e.nombre, r.nombre_rol  
    FROM Empleados e  
    JOIN Roles r ON e.id_rol = r.id_rol  
    WHERE e.correo = p_correo;  
ELSE  
    SELECT 'Login fallido' AS mensaje;  
END IF;  
END$$
```

DELIMITER ;

Paso 15: Usar el procedimiento → simulación de login

```
-- login exitoso
```

```
CALL LoginUsuario('maria@fabrica.com', 'maria456');
```

```
-- login fallido
```

```
CALL LoginUsuario('carlos@fabrica.com', 'claveMala');
```

Paso 16: Consultar el log de accesos

```
SELECT * FROM LogAccesos ORDER BY fecha_hora DESC;
```

A tener en cuenta:

- 1) No se pueden usar **triggers con SELECT**, por eso se usa un **procedure**.
- 2) Se registra el intento independientemente del resultado.
- 3) Se puede extender para agregar IP, agente de usuario, etc.
- 4) Se puede bloquear usuarios tras varios intentos fallidos (nivel avanzado).

Script Completo – Sistema de acceso seguro con hash y auditoría en MySQL

```
-- PASO 1: Crear base de datos
```



```
DROP DATABASE IF EXISTS ProduccionIndustrial;  
CREATE DATABASE ProduccionIndustrial;  
USE ProduccionIndustrial;
```

-- PASO 2: Crear tabla de roles

```
CREATE TABLE Roles (  
    id_rol INT AUTO_INCREMENT PRIMARY KEY,  
    nombre_rol VARCHAR(50)  
) ENGINE=InnoDB;
```

-- PASO 3: Insertar roles

```
INSERT INTO Roles(nombre_rol) VALUES  
( 'Operario'),  
( 'Ingeniero de Proceso'),  
( 'Técnico en Seguridad'),  
( 'Empaquetador');
```

-- PASO 4: Crear tabla de empleados con clave foránea y estado

```
CREATE TABLE Empleados (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(100),  
    correo VARCHAR(100) UNIQUE,  
    password CHAR(64),  
    id_rol INT,  
    estado ENUM('activo','inactivo') DEFAULT 'activo',  
    FOREIGN KEY (id_rol) REFERENCES Roles(id_rol)  
) ENGINE=InnoDB;
```

-- PASO 5: Insertar empleados con roles y contraseñas cifradas

```
INSERT INTO Empleados(nombre, correo, password, id_rol,  
estado) VALUES  
( 'Juan Rodríguez', 'juan@fabrica.com', SHA2('juan123', 256), 1,  
'activo'),  
( 'María Gómez', 'maria@fabrica.com', SHA2('maria456', 256), 2,  
'activo'),  
( 'Carlos Díaz', 'carlos@fabrica.com', SHA2('seguridad789', 256), 3,  
'activo'),  
( 'Laura Fernández', 'laura@fabrica.com', SHA2('laura321', 256), 4,  
'activo');
```

-- PASO 6: Crear tabla de permisos

```
CREATE TABLE Permisos (  
    id_permiso INT AUTO_INCREMENT PRIMARY KEY,  
    descripcion VARCHAR(100)  
) ENGINE=InnoDB;
```

-- PASO 7: Insertar permisos

```
INSERT INTO Permisos(descripcion) VALUES  
( 'Ver tareas asignadas'),  
( 'Modificar procesos'),
```

('Ver alertas'),
('Registrar incidentes'),
('Ver productos a embalar');

-- PASO 8: Crear tabla intermedia RolPermiso

```
CREATE TABLE RolPermiso (  
    id_rol INT,  
    id_permiso INT,  
    FOREIGN KEY (id_rol) REFERENCES Roles(id_rol),  
    FOREIGN KEY (id_permiso) REFERENCES Permisos(id_permiso)  
) ENGINE=InnoDB;
```

-- PASO 9: Asignar permisos a roles

-- Operario

```
INSERT INTO RolPermiso(id_rol, id_permiso) VALUES (1, 1);
```

-- Ingeniero de Proceso

```
INSERT INTO RolPermiso(id_rol, id_permiso) VALUES (2, 1), (2, 2);
```

-- Técnico en Seguridad

```
INSERT INTO RolPermiso(id_rol, id_permiso) VALUES (3, 3), (3, 4);
```

-- Empaquetador

```
INSERT INTO RolPermiso(id_rol, id_permiso) VALUES (4, 5);
```

-- PASO 10: Consultar permisos de un usuario (ejemplo María)

```
SELECT e.nombre AS empleado, r.nombre_rol, p.descripcion AS  
permiso  
FROM Empleados e  
JOIN Roles r ON e.id_rol = r.id_rol  
JOIN RolPermiso rp ON r.id_rol = rp.id_rol  
JOIN Permisos p ON rp.id_permiso = p.id_permiso  
WHERE e.correo = 'maria@fabrica.com';
```

-- PASO 11: Simulación de login correcto

```
SELECT e.id, e.nombre, r.nombre_rol  
FROM Empleados e  
JOIN Roles r ON e.id_rol = r.id_rol  
WHERE e.correo = 'maria@fabrica.com'  
AND e.password = SHA2('maria456', 256)  
AND e.estado = 'activo';
```

-- PASO 12: Simulación de login fallido

```
SELECT e.id, e.nombre, r.nombre_rol  
FROM Empleados e  
JOIN Roles r ON e.id_rol = r.id_rol  
WHERE e.correo = 'carlos@fabrica.com'  
AND e.password = SHA2('claveMala', 256)  
AND e.estado = 'activo';
```

-- PASO 13: Crear tabla de log de accesos

```
CREATE TABLE LogAccesos (  
    id_log INT AUTO_INCREMENT PRIMARY KEY,  
    correo VARCHAR(100),  
    resultado VARCHAR(10), -- exitoso o fallido  
    fecha_hora DATETIME DEFAULT NOW()  
) ENGINE=InnoDB;
```

-- PASO 14: Crear procedimiento almacenado para login con registro en LogAccesos
DELIMITER \$\$

```
CREATE PROCEDURE LoginUsuario(  
    IN p_correo VARCHAR(100),  
    IN p_password VARCHAR(100)  
)  
BEGIN  
    DECLARE v_contador INT;  
  
    -- Verificar existencia y estado del usuario  
    SELECT COUNT(*) INTO v_contador  
    FROM Empleados  
    WHERE correo = p_correo  
    AND password = SHA2(p_password, 256)  
    AND estado = 'activo';  
  
    -- Registrar en log  
    INSERT INTO LogAccesos(correo, resultado)  
    VALUES (p_correo, IF(v_contador > 0, 'exitoso', 'fallido'));  
  
    -- Mostrar resultado  
    IF v_contador > 0 THEN  
        SELECT e.nombre, r.nombre_rol  
        FROM Empleados e  
        JOIN Roles r ON e.id_rol = r.id_rol  
        WHERE e.correo = p_correo;  
    ELSE  
        SELECT 'Login fallido' AS mensaje;  
    END IF;  
END$$
```

DELIMITER ;

-- PASO 15: Simular login exitoso usando el procedimiento
CALL LoginUsuario('maria@fabrica.com', 'maria456');

-- PASO 16: Simular login fallido usando el procedimiento
CALL LoginUsuario('carlos@fabrica.com', 'claveMala');

-- Ver accesos registrados

```
SELECT * FROM LogAccesos ORDER BY fecha_hora DESC;
```